



ADVAITA VOICE HUB

Vedic Ashram Management, Operational Automation & Spiritual Analytics Ecosystem

PRODUCTION EDITION — VERSION 12.0.0

Lead Engineer & Architect	Gian Juti Tripura (gianjuti.csecu@gmail.com)
Academic Affiliation:	Dept. of Computer Science & Engineering, University of Chittagong
Spiritual Partner:	Advaita VOICE & ISKCON Youth Forum (IYF)
Current Release Package:	Advaita-VOICE-Hub-v12.apk (Android Target SDK 34)
Deployment Targets:	Cross-Platform Progressive Web App (PWA) & Standalone Android APK
Date of Publication:	September 2026

"Advaita VOICE Hub completely digitizes ashram operations for the University of Chittagong spiritual community. Incorporating deterministic cyclical seva rotation, emergency difficulty-balanced allocations, real-time sadhana & disciplinary auto-strikes, and dynamic dining hall financial ledgers."

Table of Contents & System Structure

1. Executive Summary & Institutional Mission

Operational context, problems solved, and enterprise ecosystem abstract.

2. High-Level System Architecture

Decoupled C4 context, React 19 view layer, Supabase PostgreSQL, and Capacitor bridge.

3. Technology Stack & Architectural Rationalization

TypeScript 6, Vite 8, Tailwind CSS, jsPDF, OneSignal, and Oxlint analysis.

4. Relational Database Schema & Data Models

Comprehensive table specifications: members, services, overrides, sadhana, and meals.

5. Core Algorithmic Engines & Formulations

Modulo rotation formula, dynamic fallback shift, emergency difficulty balancing, and auto-strikes.

6. Complete Functional Module Breakdown

Deep dive into all 16+ pages including HubHome, Seva Roster, Sadhana, Meals, and Library.

7. Security, Governance & Role-Based Access

Authentication, 5-tier access model (ADMIN, MANAGER, COUNSELOR, MEMBER), and RLS.

8. Cross-Platform Native Android & PWA

Capacitor 6 native plugins, hardware haptics, Workbox caching, and offline sync.

9. Build Automation, CI/CD & Deployment

Production Vite bundling, Gradle APK compilation, and Vercel cloud distribution.

10. WhatsApp Unicode Broadcasting Standards

Standardized Unicode announcement templates for daily rosters and verdict reports.

11. Troubleshooting, Operations & FAQ

Operational edge cases: extreme absences, account linking, leap years, and guest billing.

1. Executive Summary & Institutional Mission

The Advaita VOICE Hub is an enterprise-grade, offline-resilient, cross-platform software ecosystem engineered to digitize, automate, and analyze operations for Advaita VOICE (Vedic Oasis for Inspiration, Culture & Education). Advaita VOICE serves as the premier university-level residential spiritual academy for students of the University of Chittagong, established in partnership with the ISKCON Youth Forum (IYF).

Operational Context & The Problem Space

Prior to the deployment of Advaita VOICE Hub, ashram operations were conducted using physical paper rosters, decentralized spreadsheets, and informal messaging:

- **Service Assignments:** A 12-day cyclical seva chart was physically posted on a bulletin board. When students fell sick, took university leave, or traveled, managers spent 30–45 minutes every evening manually adjusting assignments and re-posting schedules.
- **Sadhana & Disciplinary Audits:** Morning wake-up punctuality (4:00 AM), sleep curfew (10:00 PM), and 16-round japa completion were tracked on paper scorecards, leading to delayed feedback and lack of objective counselor oversight.
- **Kitchen & Prasad Management:** Daily meal consumption fluctuated wildly due to class schedules and guests. Calculating a fair split of grocery and gas cylinder expenses at month-end led to arithmetic disputes.
- **Academic & Spiritual Curricula:** Tracking 854 syllabus topics across 6 progressive tiers and coordinating 13 residential youth camps lacked a centralized, searchable database.

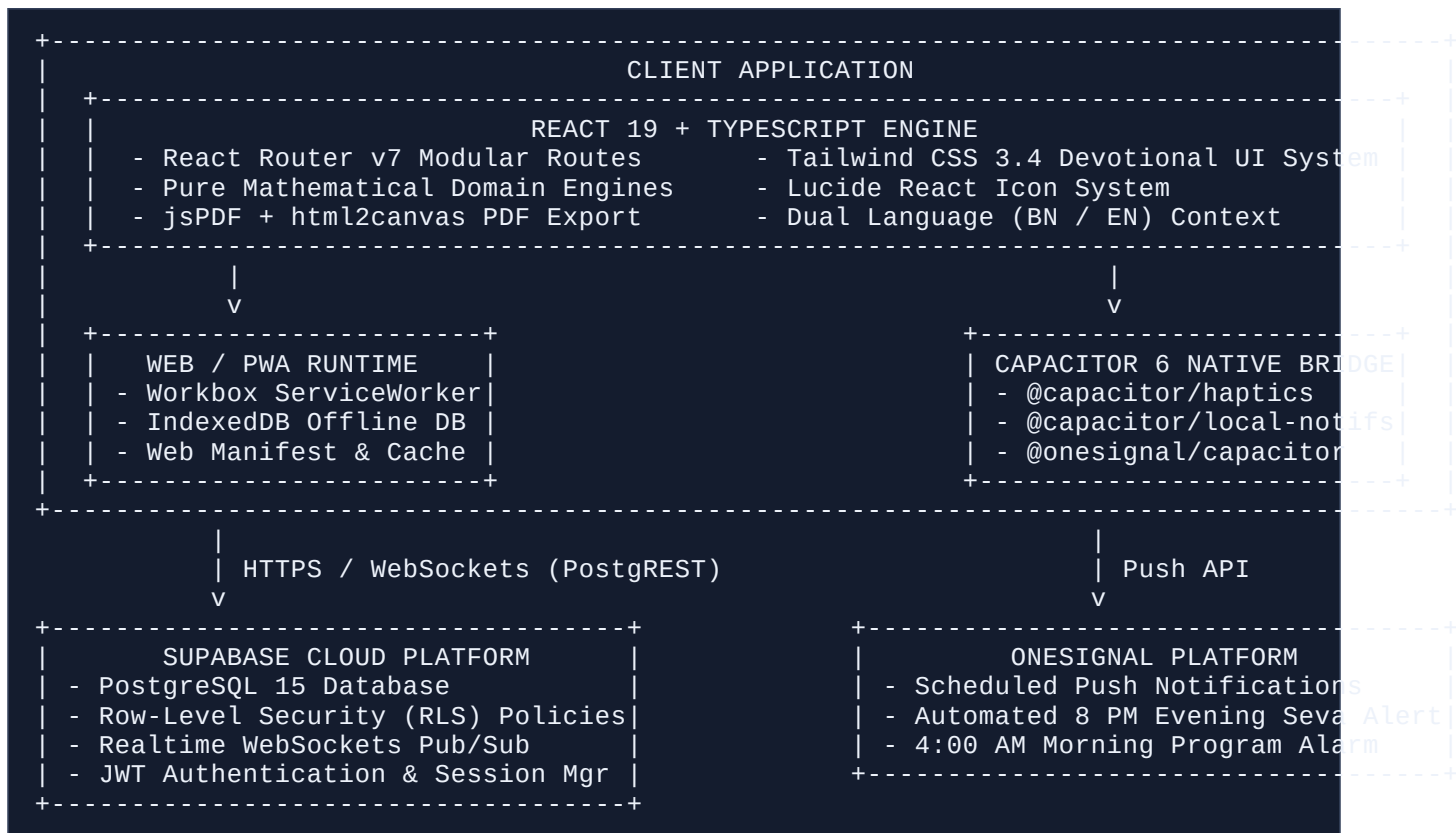
The Solution: Advaita VOICE Hub Ecosystem

Advaita VOICE Hub integrates all operational verticals into a unified, high-performance web and mobile application. Built using React 19, TypeScript 6, Vite 8, Tailwind CSS, Supabase (PostgreSQL), and Capacitor 6, the system delivers:

- 100% Automated Deterministic Seva Scheduling with zero database scheduling bloat.
- Emergency Difficulty-Balanced Service Allocation for small-contingent operations (≤ 6 members).
- Live Disciplinary Strike Engine with real-time curfew and punctuality audits.
- Dynamic Prorated Meal-Rate Engine with audit-ready client-side PDF billing statements.
- Instant One-Tap WhatsApp Unicode Broadcasting for seamless community coordination.
- Full Native Android Parity with hardware haptics, local alarms, and OneSignal push alerts.

2. High-Level System Architecture

The Advaita VOICE Hub operates on a modern, decoupled client-server architecture engineered for high availability and offline resilience:



3. Technology Stack & Architectural Rationalization

Layer	Technology	Version	Architectural Rationalization
View Layer	React	19.2.8	Concurrent rendering, declarative state management, zero memory leaks with strict memoization (useMemo, useCallback).
Language	TypeScript	6.0.2	Strict compile-time type safety preventing runtime null/undefined crashes across complex financial and scheduling engines.
Bundler & Dev Server	Vite	8.2.0	Sub-50ms Hot Module Replacement (HMR) and optimized Rollup code-splitting for minimal initial bundle size.
Styling	Tailwind CSS	3.4.17	Utility-first CSS with zero runtime overhead, responsive micro-breakpoints, and dark/light dynamic glassmorphism styling.
Database & Auth	Supabase (PostgreSQL 15)	2.112.2	Relational integrity, built-in Row-Level Security (RLS), real-time replication, and serverless PostgREST API endpoints.
Mobile Runtime	Capacitor	6.0.0	Direct native JavaScript bridge to Android SDK without Cordova overhead; maintains web code parity with native performance.
Push Notifications	OneSignal + Local	1.0.4	Guaranteed delivery of time-critical ashram announcements, seva reminders, and wake-up alarms.
PDF Statement Export	jsPDF + html2canvas	4.2.1 / 1.4.1	In-browser client-side rendering of audit-grade financial statements without requiring server-side compute resources.
Advaita VOICE Technology Division • University of Chittagong			Page 5 of 16
Static Code Quality	Online	1.75.0	Ultra-fast Post-based

4. Relational Database Schema & Data Models

The system data layer is hosted on PostgreSQL 15 via Supabase. All tables enforce relational foreign keys, timestamp auditing, and Row-Level Security (RLS):

Table: members

Stores core identity, spiritual credentials, and academic details for all ashram residents.

- id (UUID, Primary Key): Unique member identifier.
- fullName (TEXT, NOT NULL): Official legal / academic name.
- phone (TEXT): Primary contact number.
- email (TEXT): Email address for account linking and notifications.
- dob (DATE): Date of birth.
- userId (UUID, FK -> auth.users): Supabase Auth UID.
- isActive (BOOLEAN, Default: true): Active ashram residency status.
- cycleOrder (INTEGER, NOT NULL): Seniority rank (0 to 11) for cyclical duty rotation.
- role (TEXT, Default: 'MEMBER'): Access level ('ADMIN', 'INTERNAL_MANAGER', 'COUNSELOR', 'MEMBER').
- groupTier (TEXT, Default: 'VOICE'): Spiritual status ('VOICE' for compliant residents, 'LOTUS' for probation).
- department (TEXT): Academic department at University of Chittagong.
- batch (TEXT): University academic batch / session.
- bloodGroup (TEXT): Blood group for ashram emergency medical registry.
- spiritualAvatar (TEXT): Selected spiritual avatar motif (e.g., 'lotus', 'peacock', 'conch').
- avatarUrl (TEXT): Custom profile photo URL.

Table: services

Defines the 12 permanent temple services, their timings, and workload difficulty ranks.

- id (TEXT, Primary Key): Service identifier ('1' to '12').
- nameEn (TEXT, NOT NULL): English service title.
- nameBn (TEXT, NOT NULL): Bengali service title.
- descEn (TEXT): Comprehensive English duties checklist.
- descBn (TEXT): Comprehensive Bengali duties checklist.
- timing (TEXT): Scheduled operational time window (e.g., '04:00 AM - 05:30 AM').
- isActive (BOOLEAN, Default: true): Operational state of service.
- difficulty (TEXT): Workload tier ('HEAVY', 'MEDIUM_HIGH', 'MEDIUM', 'LIGHT_MEDIUM', 'LIGHT').
- weight (NUMERIC): Numerical difficulty weight (1.0 to 3.0) for workload equalization.

Table: assignment_overrides

Records daily manager interventions, absences, and replacement substitutions.

- id (UUID, Primary Key): Unique override record ID.
- dateStr (TEXT, NOT NULL): Target calendar date ('YYYY-MM-DD') or 'CONTINUOUS'.
- memberId (UUID, FK -> members.id): Resident being overridden.
- serviceId (TEXT, FK -> services.id): Assigned service ID.
- status (TEXT, NOT NULL): Status flag ('ACTIVE', 'ABSENT', 'REPLACED').
- absenceReason (TEXT): Justification (e.g., 'University Exam', 'Fever', 'Home Visit').
- replacementMemberId (UUID, FK -> members.id): Substituted devotee ID.
- managerId (UUID, FK -> members.id): Manager who authorized the change.
- timestamp (TIMESTAMPTZ): Audit log creation timestamp.

Table: sadhana_entries

Captures daily spiritual discipline metrics and calculated compliance scores.

- id (UUID, Primary Key): Unique entry ID.
- devoteeId (UUID, FK -> members.id): Devotee logging sadhana.
- dateStr (TEXT, NOT NULL): Date of record ('YYYY-MM-DD').
- wakeUpTime (TEXT): Clock time of rising (Target: <= 04:00 AM).
- wokeUpOnTime (BOOLEAN): Wake-up compliance flag.
- sleepTime (TEXT): Bedtime clock time (Curfew: <= 10:00 PM).
- sleptOnTime (BOOLEAN): Curfew compliance flag.
- morningProgramOnTime (BOOLEAN): Mangalarati punctuality.
- mangalaratiAttended (BOOLEAN): Full Mangalarati presence.
- morningClassAttended (BOOLEAN): Srimad-Bhagavatam lecture presence.
- japaBefore630 (INTEGER): Japa rounds completed before 06:30 AM.
- japaBefore830 (INTEGER): Japa rounds completed before 08:30 AM.
- japaBefore1000 (INTEGER): Japa rounds completed before 10:00 PM.
- readingMinutes (INTEGER): Minutes spent studying Srila Prabhupada's books.
- hearingMinutes (INTEGER): Minutes spent hearing spiritual discourses.
- calculatedScore (INTEGER): Normalized daily score (0 to 100).

Table: meal_overrides

Logs daily dining hall headcount and guest count multipliers.

- id (UUID, Primary Key): Meal record ID.
- memberId (UUID, FK -> members.id): Resident member ID.
- date (DATE, NOT NULL): Calendar date.

- breakfast (BOOLEAN, Default: true): Morning prasad consumption.
- lunch (BOOLEAN, Default: true): Afternoon prasad consumption.
- dinner (BOOLEAN, Default: true): Evening prasad consumption.
- guestCount (INTEGER, Default: 0): Extra guest meals consumed.
- isEkadashiFasting (BOOLEAN, Default: false): Fasting exemption flag.

Table: bazar_expenses

Records all grocery purchases, fuel costs, and kitchen operational expenditures.

- id (UUID, Primary Key): Expense record ID.
- date (DATE, NOT NULL): Purchase date.
- month (TEXT, NOT NULL): Billing cycle identifier ('YYYY-MM').
- buyer (TEXT, NOT NULL): Name of purchaser devotee.
- category (TEXT): Expense category ('VEGETABLES', 'GRAINS', 'DAIRY', 'SPICES', 'FUEL').
- items (TEXT): Itemized description of goods purchased.
- amount (NUMERIC, NOT NULL): Total cost in Bangladeshi Taka (BDT ₳).

Table: payments

Tracks financial deposits made by residents toward their monthly kitchen dues.

- id (UUID, Primary Key): Deposit transaction ID.
- memberId (UUID, FK -> members.id): Depositor devotee ID.
- amount (NUMERIC, NOT NULL): Amount deposited in BDT ₳.
- date (DATE, NOT NULL): Transaction date.
- month (TEXT, NOT NULL): Target billing month ('YYYY-MM').
- method (TEXT): Payment channel ('CASH', 'BKASH', 'NAGAD', 'BANK').
- trxId (TEXT): Electronic transaction reference ID.

5. Core Algorithmic Engines & Mathematical Formulations

5.1 Deterministic 12-Day Cyclical Seva Rotation

The ashram operates on a 12-day rotational seva schedule. Rather than generating and storing millions of static rows in the database, the system determines assignments deterministically at runtime:

$$rawServiceIndex = \big((cycleOrder + dayOfMonth) + 3\big) \bmod N + 1$$

Where:

- $cycleOrder \in [0, 11]$ is the permanent seniority index assigned to each resident devotee.

- $\text{dayOfMonth} \in [1, 31]$ is the current calendar day of the month.
- N is the total count of active services (typically 12).
- $+3$ is an ashram harmonic phase constant synchronizing the digital schedule with historical traditions.

Key Advantage: Because modulo arithmetic calculates daily assignments instantly, modifying member seniority or adding a new active service adapts the entire calendar instantaneously without needing database migrations.

5.2 Hierarchical Absence Handling & The Dynamic Shift Algorithm

When a resident cannot perform their assigned service, the manager flags them as ABSENT (single-day) or CONTINUOUS (multi-day university leave). The engine dynamically resolves missing service coverage using a priority cascade:

$\text{Fallback Priority Vector} = [10, 4, 6, 11, 2, 8, 7]$

Algorithm Execution Flow:

Detect Unattended Services: The engine compiles all services assigned to members with status `=== 'ABSENT'`.

Assemble Available Replacements Pool: The fallback priority list is scanned in order. Any member who is also absent on the target date is strictly filtered out.

Dynamic Shift Execution:

- The 1st missing service is assigned to the 1st available fallback devotee.
- The 2nd missing service is assigned to the 2nd available fallback devotee.
- The cascade continues sequentially until all services are covered.

Guaranteed Uptime: No deity or morning worship service is ever left abandoned, even during concurrent cascading absences.

5.3 Emergency 12-Service Engine with Database-Driven Difficulty Balancing

When resident headcount drops to $M \leq 6$ (during vacation or examination periods), standard 1-to-1 allocation leaves 6 or more services unassigned. The Emergency Engine combines time-relevant duty bundling and greedy workload equalization:

Workload Difficulty Weights:

- **HEAVY** (Weight: 3.0): Morning Kitchen Cooking (9), Night Dinner & Heavy Pots (10), Evening Cooking & Shayan Puja (12).
- **MEDIUM_HIGH** (Weight: 2.5): Breakfast Service & Hall Clean (5), Lunch Service & Utensils (7).
- **MEDIUM** (Weight: 2.0): Pujari Arati (1), Bhogo Offering (2), Utensils & Kirton (3).
- **LIGHT_MEDIUM** (Weight: 1.5): Kitchen Pre-Prep (4), Veg Cut & Wash (6).
- **LIGHT** (Weight: 1.0): Ashram Grounds Cleaning (8), Evening Veg Prep (11).

Time-Relevant Duty Bundles:

Services are grouped into non-overlapping temporal pairs so no single resident is scheduled for conflicting simultaneous rituals (e.g., Morning Cook + Evening Veg Prep, or Pujari Arati + Grounds Cleaning).

Workload Variance Minimization:

$$T = \frac{\sum_{i=1}^{12} w(s_i)}{M}$$

$$\min \sum_{j=1}^M (W_j - T)^2$$

The engine ensures all 12 services are executed daily while maintaining equitable physical workload distribution across present residents.

5.4 Live Disciplinary Auto-Strike State Machine & Rule Infractions

The disciplinary engine continuously audits daily sadhana logs against mandatory ashram guidelines:

- Rule 1: Sleep Curfew: Must sleep at or before 10:00 PM. Violation: \$+1\$ Infraction.
- Rule 2: Wake-Up Punctuality: Must wake up at or before 04:00 AM. Violation: \$+1\$ Infraction.
- Rule 3: Morning Program Attendance: Must be present at 04:30 AM Mangalarati. Violation: \$+1\$ Infraction.
- Rule 4: Srimad-Bhagavatam Class: Must attend daily morning class. Violation: \$+1\$ Infraction.

Monthly Strike Escalation Tiers:

- 0 Strikes (Green): Exemplary Sadhana Status (100% Compliance).
- 1 Strike (Yellow): Warning Tier 1.
- 2 Strikes (Orange): Warning Tier 2 — Strict Probation.
- 3+ Strikes (Red): Automatic degradation from VOICE resident tier to LOTUS probation tier; triggers automated notification to the Ashram Counselor.

5.5 Dynamic Prorated Meal Rate & Kitchen Bazar Accounting

The kitchen finances operate on a zero-profit, actual-cost cooperative split model:

$$\text{Meal Rate} = \frac{\text{Total Monthly Bazar Expenses}}{\text{Total Monthly Meals Consumed}}$$

$$\text{Member Liability} = (\text{Member Meals Consumed} + \text{Guest Meals}) \times \text{Meal Rate}$$

$$\text{Final Balance} = \text{Opening Balance} + \sum \text{Payments} - \text{Member Liability}$$

- Positive Final Balance: Devotee has advance credit carrying forward to the next month.

- Negative Final Balance: Devotee has outstanding dues requiring payment reconciliation.

6. Complete Functional Module Breakdown

6.1 Central Command Hub (HubHome.tsx)

The application entry point features:

- Quick Portal Launcher: A 5×3 fixed-size rectangular grid providing one-tap navigation to all primary verticals.
- Daily Seva Snapshot: Displays today's assigned duty for the logged-in member.
- Spiritual Quote & Ticker: Dynamic daily scriptural verse and community announcement banners.
- Atmospheric Theme Aura: Subtle Sri Krishna Vrindavan background imagery and dynamic glow.

6.2 Service Cycle & Member Seva Dashboard (MemberDashboard.tsx)

- Personal Duty Card: Displays today's and tomorrow's assigned seva duties with timings and checklist of responsibilities.
- Replacement Notification: Alerts the member if they have been drafted as a replacement for an absent devotee.
- Account Self-Linking: Allows newly authenticated users to claim their profile from the database registry.

6.3 Manager Command Dashboard & Emergency Roster (ManagerDashboard.tsx, EmergencyRosterPage.tsx)

- Live 12-Service Roster: Overview of all active temple duties with live status badges (● PRESENT, ● ABSENT, ➡ REPLACED).
- Absence & Override Controls: Manager modal to log emergency absences, input reasons, and trigger automatic fallback cascades.
- Emergency Chart Mode: Dedicated interface for small-group periods (<= 6 members) with workload balance meters and duty bundles.
- One-Tap WhatsApp Export: Generates ready-to-paste Unicode announcements formatted for community WhatsApp groups.

6.4 Sadhana Tracker & 16-Round Japa Milestones (SadhanaTracker.tsx)

- Interactive Scorecard: Easy-tap inputs for wake-up time, sleep time, Mangalarati attendance, class presence, and book reading minutes.
- Three-Stage Japa Milestones: Tracks rounds completed before 06:30 AM (Morning Golden Hours), before 08:30 AM (Post-Program), and before 10:00 PM (Curfew).
- Historical Trend Charts: 7-day and 30-day compliance graphs showing performance consistency.

6.5 Counselor Desk & Disciplinary Audit (CounselorDesk.tsx, AshramDisciplineAudit.tsx)

- Counselor Oversight Portal: Allows ashram directors to review student batches, inspect sadhana scores, and record mentorship notes.
- Automated Strike Audit Board: Displays monthly infraction tallies, strike indicators, and members subject to LOTUS group degradation.
- Discipline WhatsApp Verdict: Formats comprehensive monthly evaluation summaries with emoji indicators for community transparency.

6.6 Dining Hall & Prasad Management Suite (MealDashboard.tsx, MealAttendancePage.tsx, BazarTrackerPage.tsx, MealPaymentsPage.tsx, MealReportsPage.tsx)

- Meal Booking Roster: Daily checkboxes for Breakfast, Lunch, and Dinner with guest count steppers and Ekadashi fasting toggles.
- Bazar Expense Tracker: Form to log grocery receipts, gas cylinders, category breakdown, and buyer identity.
- Payment Deposit Ledger: Financial log recording bKash, Nagad, Cash, and Bank transactions with electronic transaction IDs.
- Monthly Audit & PDF Statement: Compiles individual attendance, meal rates, and balances into client-side vector PDF statements.

6.7 Devotee Master Directory & Academic Profiles (DevoteeProfilesPage.tsx, DevoteeProfileModal.tsx)

- Complete Devotee Registry: Searchable directory filterable by university department, batch, blood group, counselor, and group tier.
- Profile Customizer Modal: Personal avatar selection (sacred spiritual motifs or custom photo upload), phone update, and bio editor.

6.8 6-Tier 854-Topic Vedic Curriculum Explorer (SyllabusExplorer.tsx, CampsPage.tsx, CoursesPage.tsx)

- 6 Progressive Educational Tiers:
 - Discover Yourself (DYS) — Foundational Vedic Philosophy
 - Spiritual Scientist — Science of Self-Realization & Cosmology
 - Positron — Applied Ethics, Mind Control & Daily Sadhana
 - Protons — Leadership, Vaishnava Etiquette & Preaching
 - Bhakti Shastri — Systematic Study of Bhagavad-gita, Nectar of Devotion, Upadesamrita, Sri Isopanisad
 - Sri Caitanya-caritamrta (CC) — Advanced Gaudiya Theology
- 4-Year VOICE Roadmap: Interactive stage progression tracking 13 residential retreats, university exams, and spiritual vows.

6.9 Sebananda Vedic E-Library & Lecture Archive (SebanandaLibrary.tsx, UnifiedLectureLibrary.tsx)

- Curated Digital Bookshelf: Searchable repository of Vedic literature, study guides, and articles with direct PDF downloads.
- Discourse Archive: Categorized audio and video lectures by visiting spiritual masters, sannyasis, and senior counselors.

6.10 Vaishnava Calendar & Festivals (VaishnavaCalendarPage.tsx)

- Complete lunar calendar tracking Ekadashi fasting dates, paran timings, Vaishnava appearance/disappearance days, and festival schedules.

6.11 Youth Camps & Leadership Retreats (CampsPage.tsx)

- Registration and syllabus outlines for the 13 residential camps (e.g., Prerana, Chetana, Sannikarsha, Sadhana Camp).

6.12 Preacher's Outreach Toolkit (PreachersToolkit.tsx)

- Follow-up tracker for prospective students, calling log records, stage progressions, and counseling templates.

6.13 Community Announcements & Notification Center (AnnouncementsPage.tsx)

- Broadcast feed for official ashram directives, emergency alerts, and festival arrangements.

6.14 Divine Theme Customizer & Sacred Brand Typography (ThemeCustomizerModal.tsx, FallingFlowers.tsx)

- Chrome-style personal palette customizer (Amber, Emerald, Rose, Indigo, Sky), light/dark mode switcher, background aura controls, and interactive canvas falling flower petals.

7. Security, Governance & Role-Based Access Control (RBAC)

The system implements a defense-in-depth security model across five architectural tiers:

TIER 1: AUTHENTICATION & SESSIONS
- Supabase JWT Token Auth with persistent local session encryption
- Password reset workflows via secure email magic links
TIER 2: ROLE-BASED ACCESS CONTROL
- ADMIN: Master configuration, service definitions, tier promotions, audits
- INTERNAL_MANAGER: Seva overrides, emergency rosters, bazar expenses
- COUNSELOR: Sadhana monitoring, discipline reviews, student care
- MEMBER: Personal sadhana entry, meal attendance, duty view
TIER 3: DATABASE ROW-LEVEL SECURITY
- PostgreSQL RLS policies restrict table mutations based on authenticated UID
- Devotees can only modify their own sadhana and meal attendance rows
TIER 4: INPUT SANITIZATION & INTEGRITY
- Strict TypeScript schema validation on all API payloads
- DOMPurify sanitization on rich announcements and user-submitted text

8. Cross-Platform Native Android & PWA Engineering

Capacitor 6 Native Android Integration

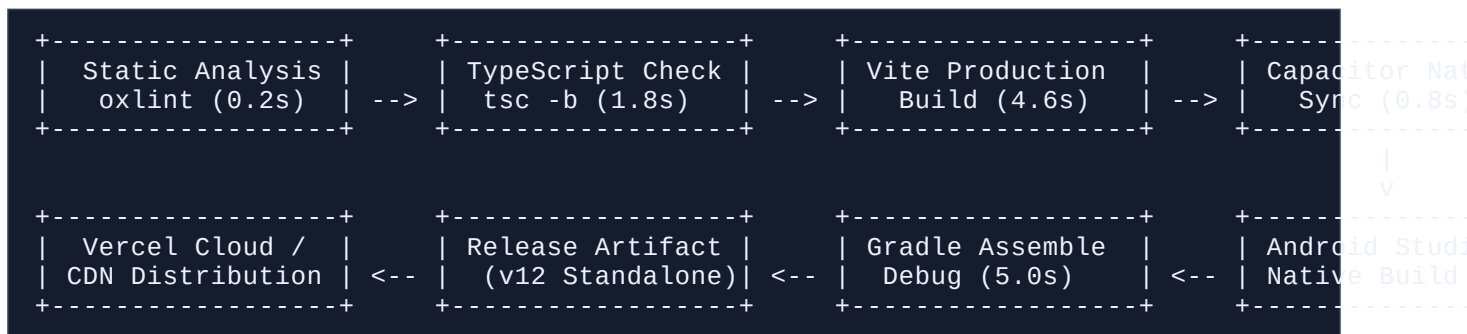
Advaita VOICE Hub compiles natively for Android (Target SDK 34, Android 14) using Capacitor 6:

- **Hardware Haptics (@capacitor/haptics):** Provides tactile vibration feedback on button presses, attendance toggling, and emergency roster assignments.
- **Scheduled Alarms (@capacitor/local-notifications):** Registers local device alarms for the 4:00 AM wake-up bell and 8:00 PM evening seva roster broadcast.
- **Cloud Push Alerts (@onesignal/capacitor-plugin):** Delivers time-sensitive push announcements sent by ashram managers.
- **Hardware Acceleration:** Native Chrome WebView rendering achieving 60fps animations.

Progressive Web App (PWA) & Offline Resilience

- **Workbox Service Workers (vite-plugin-pwa):** Pre-caches 87+ core static assets (2.49 MB) for instantaneous cold starts.
- **IndexedDB Local Database (localDb.ts):** All seva schedules, member profiles, and sadhana logs are cached locally on device.
- **Last-Write-Wins (LWW) Synchronization:** When internet connectivity resumes, offline modifications sync with Supabase using UTC timestamps.

9. Build Automation, CI/CD & Deployment Pipeline



Production Build Commands

```
# 1. Clean lint check
npm run lint

# 2. Compile TypeScript and build production web assets
npm run build

# 3. Synchronize with native Android project
npx cap sync android

# 4. Compile standalone debug APK via Gradle
cd android
./gradlew assembleDebug
```

The compiled package is output to android/app/build/outputs/apk/debug/app-debug.apk and distributed as Advaita-VOICE-Hub-v12.apk.

10. WhatsApp Unicode Broadcasting Standards

The application formats all community announcements with standardized Unicode emojis and bold markdown headers:

Sample Daily Seva Roster Broadcast

```
🌟 *ADVAITA VOICE ASHRAM SEVA ROSTER* 🌟
📅 *Date:* 13 September 2026 (Sunday)
🏛️ *Temple Services & Devotee Assignments:*

🟢 *01. Pujari Mangalarati:* Gian Juti Tripura
🟢 *02. Deity Bhogo Offering:* Dipen P.
🟢 *03. Utensils & Mangal Kirton:* Subrata D.
🟢 *04. Kitchen Pre-Prep:* Chinmoy B.
🟢 *05. Breakfast Service & Hall:* Joydeb P.
🟢 *06. Vegetable Cutting:* Animesh S.
🔴 *07. Lunch Service:* Gouranga D. (ABSENT - University Exam)
↳ 🔄 *REPLACED BY:* Subrata D. [Service 10 Fallback]
🟢 *08. Ashram Grounds:* Niloy T.
🟢 *09. Morning Cooking:* Pranta G.
🟢 *10. Night Dinner & Pots:* Subrata D.
🟢 *11. Evening Veg Prep:* Bishal D.
🟢 *12. Night Cooking & Shayan:* Rupam D.

📢 *Notice:* Devotees must report to assigned seva 10 minutes prior to scheduled timing.
_Advaita VOICE Technology Division_
```

11. Troubleshooting, Operations & Frequently Asked Questions

FAQ 1: What happens if more than 7 members are absent simultaneously?

- Answer: If the 7-person fallback priority pool is exhausted, the engine displays an alert on the Manager Dashboard prompting the manager to activate the Emergency Chart Mode, which packages all 12 services into balanced time bundles for the remaining devotees.

FAQ 2: How does a new resident connect their login to their member profile?

- Answer: Upon logging in, an unlinked resident is redirected to the Member Dashboard where an Account Linking dropdown appears containing all unclaimed profiles. Selecting their name permanently associates their Supabase Auth UID with that record.

FAQ 3: How does the system handle leap years and month transitions?

- Answer: The engine derives month length via `new Date(year, month, 0).getDate()`. Transitions from the 31st to the 1st rotate seamlessly without resetting seniority indices or producing negative modulo offsets.

FAQ 4: How are meal rates calculated when guests eat at the ashram?

- Answer: Guests are added as an integer multiplier in `guestCount`. The total meals divisor increases accordingly, ensuring that members who host guests pay for their guests' meals without increasing the meal rate for other residents.

Certified Specification

Author: Gian Juti Tripura

Lead Engineer & Architect

B.Sc. in Computer Science & Engineering, University of Chittagong

Advaita VOICE Technology Division, ISKCON Youth Forum